AULA 1 · TEORIA

Teste de Software: fundamentos e planejamento

Técnico em Desenvolvimento de Sistemas
UC Teste de Software - visão essencial



Por que testar?

- Testar = avaliar a qualidade e dar confiança de que o software faz o que deve
- **Três objetivos:** confiança, funcionalidade, performance
- Achar o defeito cedo custa menos do que corrigir depois do lançamento
- Qualidade não é acidente: é esforço de toda a equipe

Sem testes, na TechSolutions…	Impacto
Inscrições duplicadas	Retrabalho (100+ horas)
QR code de presença falha	Multas contratuais
Pagamentos não processados	Chargebacks (R$ 150 mil+)

Fonte: livro, quadro de riscos do desafio inicial

Glossário essencial: erro, defeito e falha

ERRO

Ação humana equivocada: o programador entende ou escreve algo errado.

→

DEFEITO (bug)

Imperfeição no código ou documento, causada pelo erro.

→

FALHA

Comportamento errado percebido quando o programa executa.

Exemplo (calculadora da Aula 2)

Esquece de tratar número gigante (erro) → o except não prevê OverflowError (defeito) → digitar 9.9^999 não mostra resultado (falha).

Também no glossário: caso de teste · roteiro de teste · critério de aceitação · teste de regressão · cobertura · teste de unidade

Tipos de teste: funcional × não funcional

FUNCIONAL - o que o sistema faz

- Entrada → saída conforme o requisito
- 'Adicionar ao carrinho' atualiza o total?
- Bloqueia acesso após 3 senhas erradas?
- Botão 'Enviar' só ativa com os campos obrigatórios?

NÃO FUNCIONAL - como o sistema faz

- Performance: página com 10.000 usuários ao mesmo tempo
- Segurança: dados criptografados, SQL injection
- Usabilidade: um idoso consegue navegar?
- Confiabilidade e portabilidade

Níveis de teste

1 · Unitário

Uma função ou classe isolada.
Quem: o desenvolvedor.
Python: pytest, unittest.

2 · Integração

Módulos juntos: formato de dados, API, banco.
Ex.: carrinho + gateway de pagamento.

3 · Sistema

Sistema completo, do início ao fim, em ambiente parecido com produção.

4 · Aceitação

O cliente valida se o software atende ao que ele precisa (UAT).

Sistema: “foi construído corretamente?” · Aceitação: “atende ao que eu preciso?”

Pirâmide: muitos testes unitários (rápidos e baratos) na base; poucos de sistema e aceitação no topo.

Técnicas de teste

Regressão

Nova função ou correção não pode quebrar o que já funcionava. Ex.: corrigiu o login, retesta a compra.

Estresse

Carga extrema para achar o limite. Ex.: servidor de jogo lotado.

Recuperação

O sistema volta após falha? Ex.: queda de energia no banco, transações revertidas?

Performance

Tempo de resposta, throughput e uso de recursos sob carga.

Segurança

Vulnerabilidades: SQL injection, XSS, negação de serviço.

Paralelo

Vários testes ao mesmo tempo em ambientes ou navegadores diferentes.

Estratégias: caixa-preta, branca e cinza

Tipo	O que testa	Exemplo	Ferramenta
Caixa-preta	Comportamento externo	Login com senha errada mostra mensagem?	Selenium ou manual
Caixa-branca	Estrutura interna: if, caminhos	calcular_desconto: um teste por caminho	pytest, SonarQube
Caixa-cinzenta	API + banco de dados	POST /cadastro com idade 17 e conferir o banco	Postman + SQL

Teste unitário em Python = caixa-branca → é o foco da Aula 2

Planejamento: análise de risco e plano de teste

Análise de risco

- Risco = probabilidade × impacto
- Teste primeiro o que mais dói: pagamento antes da cor do botão
- TechSolutions: pagamentos, QR code de presença, certificados

Plano de teste

- Escopo: o que será e o que não será testado
- Estratégia e tipos de teste
- Recursos e ambiente
- Cronograma
- Critérios de aceite (ex.: 90% aprovados e nenhum defeito crítico)

Caso de teste: anatomia

Campo	Exemplo
ID	TC_LOGIN_01
Título	Login com credenciais válidas
Pré-condição	Usuário “teste” cadastrado
Dados	usuário: teste · senha: senha123
Passos	1 Abrir o login · 2 Preencher · 3 Entrar
Resultado esperado	Vai para a página principal
Prioridade / Status	Alta / Não executado

Tipos de caso

- Positivo: entrada válida
- Negativo: entrada inválida ou inesperada
- Borda: nos limites
- Cenário: jornada do usuário

Como criar bons casos de teste

Partição de equivalência

Agrupe entradas que se comportam igual e teste um de cada grupo.
Idades: 0–17 | 18–130 | inválidas.

Valor limite

Teste a fronteira de cada grupo.
Campo de até 50 caracteres: 0, 1, 49, 50 e 51.

Tabela de decisão

Condições × ações.
Duas condições (sim/não) = 4 combinações = 4 testes.

Idade ≥ 18? Teste 17, 18 e 19 — a maioria dos bugs mora na borda.

Execução: ambiente, equipe, ferramentas e defeitos

Ambiente

- Parecido com produção
- Isolado e reproduzível
- Versões registradas

Equipe

- Papéis definidos: líder, analista, testador, automatizador
- Qualidade é de todos

Ferramentas

- Gestão de teste: TestLink, Zephyr, TestRail
- Defeitos: Jira, Bugzilla, Redmine
- Automação: Selenium, Postman, pytest

Defeito: Aberto → Em análise → Em desenvolvimento → Corrigido → Fechado

Severidade = impacto no sistema · Prioridade = urgência da correção

Relatório de teste e normalização

Taxa de aprovação = aprovados ÷ executados × 100

Exemplo do livro	Total	%
Casos executados	120	100
Aprovados	102	85
Reprovados	15	12,5
Bloqueados	3	2,5

Meta 90% e 2 defeitos críticos abertos → decisão: reprovar o release (NO-GO).

Normalização

- Nomes padrão: TC_[Módulo]_[Funcionalidade]_[Cenário]
- Status padrão: não executado, aprovado, reprovado, bloqueado
- Um modelo único de caso de teste